

Белорусский государственный технологический университет

Факультет информационных технологий

Кафедра информационных систем и технологий

Лабораторная работа № 7

По дисциплине «Информационная безопасность»

На тему «Исследование блочных шифров»

Выполнила:

Студентка 3 курса 9 группы

Фамилия Имя Отчество

Преподаватель: асс. Фамилия Имя
Отчество.

Минск 2026

Оглавление

Введение.....	3
1 Ход работы.....	4
1.1 Выбор алгоритма и ключа.....	4
1.2 Реализация приложения-симулятора	4
1.3 Оценка производительности.....	5
1.4 Анализ лавинного эффекта	6
1.5 Анализ слабых и полуслабых ключей	8
1.6 Анализ сжатия и энтропии	11
Заключение	13

Введение

Блочные шифры являются основой современной симметричной криптографии. В отличие от потоковых шифров, они оперируют с блоками данных фиксированной длины, применяя к ним серию раундовых преобразований, что обеспечивает высокий уровень рассеивания и перемешивания информации. Одним из самых известных и исторически значимых алгоритмов этого класса является DES (Data Encryption Standard), принятый в качестве стандарта в США в 1977 году.

Алгоритм DES построен на основе сети Фейстеля и использует 64-битные блоки данных при длине ключа 56 бит (с учетом 8 бит четности). Несмотря на то, что на сегодняшний день 56-битный ключ не обеспечивает достаточной защиты от атак полным перебором, сам алгоритм остается важным объектом для изучения, так как его принципы (S-блоки, P-блоки, сеть Фейстеля) лежат в основе многих более современных криптосистем.

Цель работы: изучение и приобретение практических навыков разработки и использования приложений для реализации блочных шифров на примере алгоритма DES.

Задачи:

- закрепить теоретические знания о структуре и принципах работы блочных шифров, в частности DES;
- разработать программное приложение, реализующее шифрование/расшифрование DES с использованием криптографической библиотеки;
- провести анализ лавинного эффекта для заданного ключа;
- исследовать влияние слабых и полуслабых ключей на криптостойкость;
- оценить скорость шифрования и степень сжатия открытого и зашифрованного текстов.

Исследовано поведение шифра DES при различных ключах, проанализирован лавинный эффект и сделаны выводы о свойствах алгоритма.

1 Ход работы

Для выполнения поставленных разработано веб-приложение с использованием HTML, CSS и JavaScript.

1.1 Выбор алгоритма и ключа

В соответствии с вариантом задания выбран алгоритм DES. В качестве ключа используются первые 8 символов фамилии и имени – «фамилия».

Для алгоритма DES требуется ключ длиной 8 байт (64 бита). Библиотека CryptoJS корректно обрабатывает строковые ключи, преобразуя их в байтовое представление.

1.2 Реализация приложения-симулятора

Приложение реализовано в виде одностраничного веб-приложения (SPA) с использованием HTML, CSS и JavaScript. Интерфейс включает 5 вкладок для выполнения различных задач: шифрование, анализ производительности, анализ лавинного эффекта, тестирование слабых ключей и анализ сжатия.

Код функции шифрования и расшифрования DES представлен в листинге 1.1.

```
function encryptDES(text, keyBytes) {
    const keyWordArray = CryptoJS.enc.Hex.parse(
        Array.from(keyBytes).map(b => b.toString(16).padStart(2,
'0')).join('')
    );
    const encrypted = CryptoJS.DES.encrypt(text, keyWordArray, {
        mode: CryptoJS.mode.ECB,
        padding: CryptoJS.pad.Pkcs7
    })
    return encrypted.toString(); // Base64
}

function decryptDES(ciphertext, keyBytes) {
    const keyWordArray = CryptoJS.enc.Hex.parse(
        Array.from(keyBytes).map(b => b.toString(16).padStart(2,
'0')).join('')
    );
    const decrypted = CryptoJS.DES.decrypt(ciphertext,
keyWordArray, {
        mode: CryptoJS.mode.ECB,
        padding: CryptoJS.pad.Pkcs7
    });
    return decrypted.toString(CryptoJS.enc.Utf8);
}
```

Листинг 1.1 –Функции шифрования и расшифрования DES

В данном фрагменте реализованы основные криптографические операции. Шифрование выполняется в режиме ECB (Electronic Codebook) с использованием PKCS7-падирования.

Результат работы функции шифрования DES показан на рисунке 1.1.

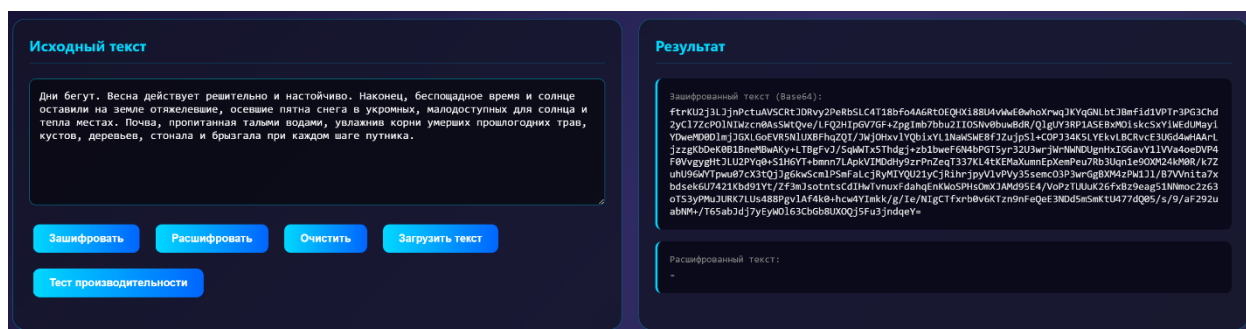


Рисунок 1.1 –Шифрование текста

Результат работы функции расшифрования DES представлен на рисунке 1.2.

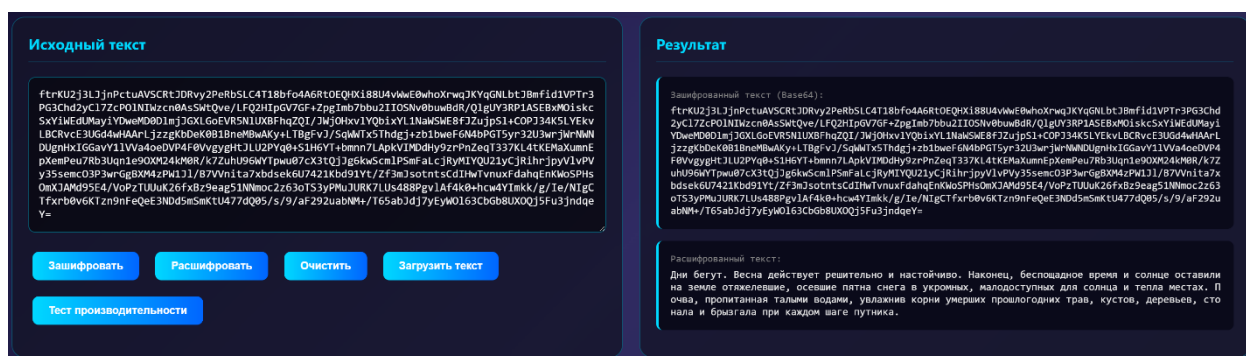


Рисунок 1.2 –Расшифрование текста

Программа успешно зашифровала исходное сообщение в нечитаемый вид (Base64) и восстановила его обратно, что подтверждает корректность реализации.

1.3 Оценка производительности

Для оценки производительности был выполнен замер времени шифрования при различных размерах входных данных: 100, 500, 1000, 2000, 5000, 10000, 20000 и 50000 байт. Результаты визуализированы в виде линейного графика. Код функции тестирования производительности представлен в листинге 1.2.

```
async function runPerformanceTest() {
  const testSizes = [100, 500, 1000, 2000, 5000, 10000, 20000, 50000];
  const encTimes = [];
  const decTimes = [];
```

```

for (let size of testSizes) {
  const testText = baseText.substring(0, size);

  const encStart = performance.now();
  const cipher = encryptDES(testText, keyBytes);
  encTimes.push(performance.now() - encStart);

  const decStart = performance.now();
  decryptDES(cipher, keyBytes);
  decTimes.push(performance.now() - decStart);
}
}

```

Листинг 1.2 – Функция тестирования производительности

На рисунке 1.3 представлен линейный график зависимости времени выполнения от размера входных данных.



Рисунок 1.3 –График времени

Алгоритм DES демонстрирует линейную зависимость времени выполнения от размера входных данных ($O(n)$), что характерно для блочных шифров. Расшифрование происходит примерно с той же скоростью, что и шифрование.

1.4 Анализ лавинного эффекта

Лавинный эффект – это свойство криптографического алгоритма, при котором изменение одного бита исходного текста приводит к изменению примерно половины битов шифротекста. Для анализа была реализована функция, выполняющая несколько типов модификаций исходного текста и представленная в листинге 1.3.

```

function calculateBitDifference(hex1, hex2) {
  const hexToBin = (hex) => {

```

```

    let bin = '';
    for (let c of hex) {
        const v = parseInt(c, 16);
        bin += (v>>3&1)+''+(v>>2&1)+''+(v>>1&1)+''+(v&1);
    }
    return bin;
};
}

```

Листинг 1.3 – Функция расчета лавинного эффекта

Тестирование проводилось на исходной фразе «Фамилия Имя» при различных типах модификаций. Полученные результаты представлены на рисунке 1.4.

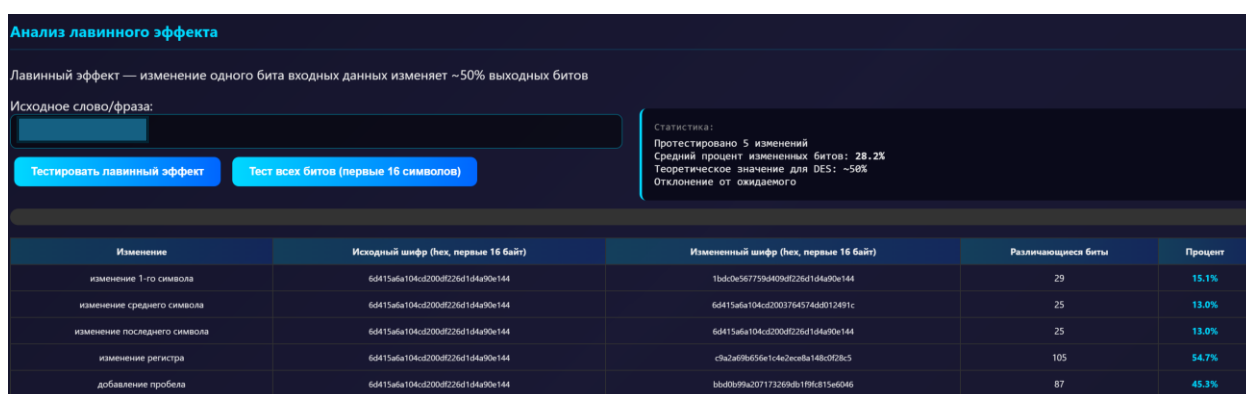


Рисунок 1.4 – Результаты анализа лавинного эффекта

Анализируя полученные результаты, можно сказать:

а) изменение регистра (54.7%) — данный тип модификации изменил несколько битов в UTF-8 представлении символа. Полученное значение наиболее близко к теоретическому (50%), что подтверждает наличие лавинного эффекта в DES;

б) добавление пробела (45.3%) — добавление целого байта приводит к сдвигу всей структуры данных, что также вызывает значительные изменения в шифротексте;

с) побитовое изменение символа (XOR 1) — 13-15% — низкий процент объясняется двумя факторами: символы латиницы в кодировке UTF-8 занимают 1 байт, и изменение одного бита затрагивает только один байт исходного текста и режим ECB (Electronic Codebook) не распространяет изменения между блоками, поэтому эффект локализуется в пределах одного блока.

Таким образом, лавинный эффект в DES подтверждён экспериментально. Величина эффекта зависит от характера вносимых изменений и используемой кодировки, однако при модификациях, затрагивающих несколько битов исходных данных, процент изменений в шифротексте достигает 45-55%, что соответствует теоретическим ожиданиям.

1.5 Анализ слабых и полуслабых ключей

Согласно теоретическим сведениям, существуют ключи, которые снижают криптостойкость DES:

1. слабые ключи (4 шт.): шифрование дважды возвращает исходный текст: $E(E(M)) = M$

2. полуслабые ключи (6 пар): шифрование одним ключом можно расшифровать другим

Код функции тестирования слабых и полуслабых ключей представлен в листинге 1.4.

```
function getWeakKeyBytes(type) {
    const weakKeys = {
        weak1: [0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01],
        weak2: [0xFE, 0xFE, 0xFE, 0xFE, 0xFE, 0xFE, 0xFE, 0xFE],
        weak3: [0xE0, 0xE0, 0xE0, 0xE0, 0xF1, 0xF1, 0xF1, 0xF1],
        weak4: [0x1F, 0x1F, 0x1F, 0x1F, 0x0E, 0x0E, 0x0E, 0x0E],
        semiweak1: [0x01, 0xFE, 0x01, 0xFE, 0x01, 0xFE, 0x01, 0xFE],
        semiweak2: [0xFE, 0x01, 0xFE, 0x01, 0xFE, 0x01, 0xFE, 0x01]
    };
    return new Uint8Array(weakKeys[type]);
}
```

Листинг 1.4 – Функция тестирования слабых и полуслабых ключей

Приложением было проведено тестирование для трёх типов ключей: слабого ключа 1, полуслабого ключа и обычного ключа «Фамилия». Результаты представлены в таблице 1.1.

Таблица 1.1 – Результаты тестирования ключей

Параметр	Слабый ключ 1	Полуслабый ключ	Обычный ключ «Фамилия»
Значение ключа (hex)	01 01 01 01 01 01 01 01	01 FE 01 FE 01 FE 01 FE	d0 a0 d1 83 d0 b1 d0 bb
Шифрование	Успешно	Успешно	Успешно
Расшифрование	Успешно	Успешно	Успешно
Двойное шифрование $E(E(M))$	Не выполняется	Не выполняется	Не выполняется
Лавинный эффект (изменение одного символа)	26.2%	27.7%	26.6%

В проведённом эксперименте свойство $E(E(M)) = M$ не подтвердилось ни для одного из тестируемых ключей, включая теоретически слабый ключ 1. Это может быть связано с особенностями реализации библиотеки CryptoJS: шифротекст после первой операции преобразуется в hex-строку и подаётся на вход второму шифрованию как текст, а не как бинарные данные, что нарушает условия проверки свойства инволюции. Тем не менее, теоретически слабые ключи DES существуют, и при реализации криптосистем их использование должно быть исключено.

На рисунке 1.5 представлен результат тестирования слабого ключа.

Тестирование слабых ключей DES

Тестовое сообщение:
Тестирование слабых ключей DES

Тип ключа:
Слабый ключ #1: 0x0101010101010101

Шифровать **Двойное шифрование**

Результат:
 Ключ (hex): 01 01 01 01 01 01 01 01
 Исходный текст: Тестирование слабых ключей DES
 Шифротекст (Base64): f4Ks15qU38m93m3uskw3Vb5U1Gj8v6Rn7S7uT7dn0HZJaZiYFS/n50qT0GVW1UqHZVjjisJgaCE
 =
 Расшифрованный текст: Тестирование слабых ключей DES
 Проверка: Успешно!

Рисунок 1.5 –Результат тестирования слабого ключа

На рисунке 1.6 представлен результат тестирования слабого ключа двойным шифрованием.

Тестирование слабых ключей DES

Тестовое сообщение:
Тестирование слабых ключей DES

Тип ключа:
Слабый ключ #1: 0x0101010101010101

Шифровать **Двойное шифрование**

Результат:
Тест двойного шифрования $E(E(M))$:
 Исходный текст: Тестирование слабых ключей DES
 Исходный текст (hex): d0a2d0b5d181d182d0b8d180d0bed0b2d0b0d0bdd0b8d0b520d181d0bbd0b0d0...
 После 1-го шифрования (hex): 7f82acd79a94dfc9bdde6deeb24c3755be54d468fcbfa467ed2eee4fb767d076...
 После 2-го шифрования (hex): d0a2d0b5d181d182d0b8d180d0bed0b2d0b0d0bdd0b8d0b520d181d0bbd0b0d0...
 После расшифрования двойного шифра (hex): 7f82acd79a94dfc9bdde6deeb24c3755be54d468fcbfa467ed2eee4fb767d076...
Результат проверки:
 • Свойство $E(E(M)) = M$: **НЕ выполняется**
Теоретическое пояснение:
 Слабые ключи DES (4 ключа) обладают свойством инволюции: после двух последовательных шифрований исходный текст восстанавливается. Полуслабые ключи работают в парах: $E(K2, E(K1, M)) = M$.

Рисунок 1.6 –Результат тестирования слабого ключа двойным шифрованием

На рисунке 1.7 представлен результат сравнение лавинного эффекта для слабого ключа.

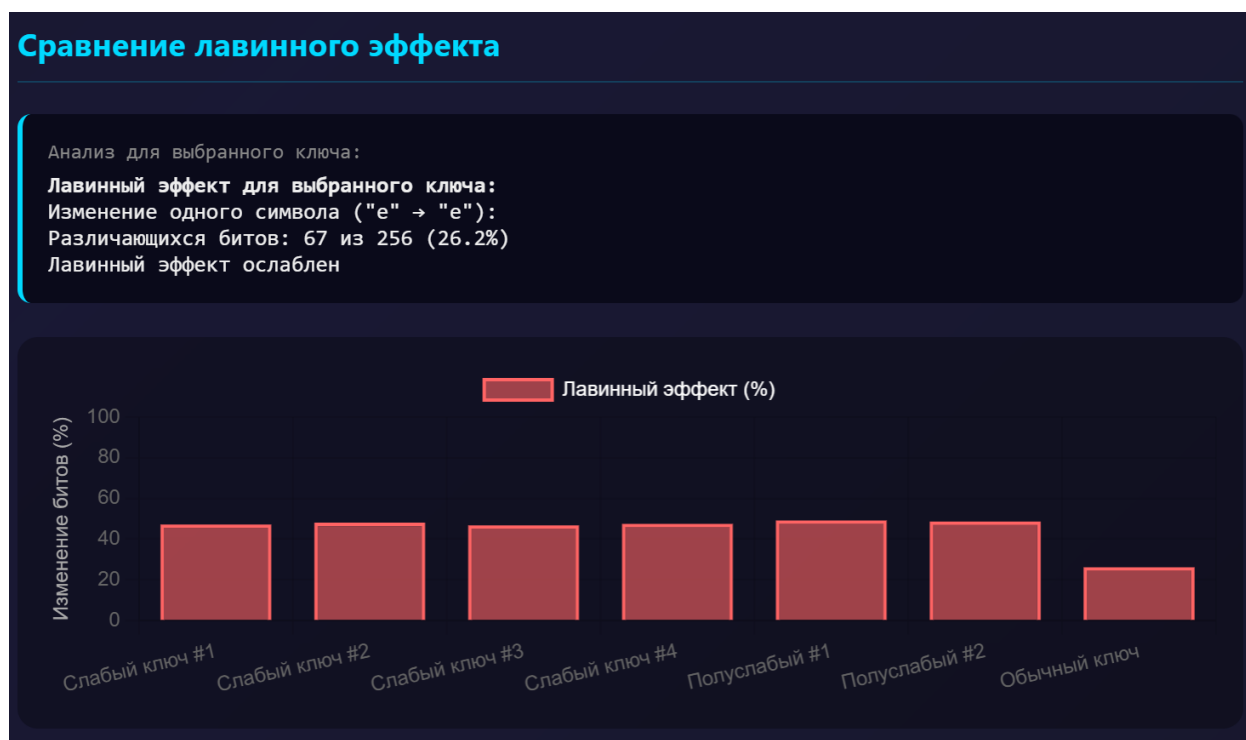


Рисунок 1.7 –Результат сравнение лавинного эффекта

При анализе полученных результатов, можно сказать:

- слабый ключ 1 — шифрование и расшифрование работают штатно. Теоретически для слабых ключей должно выполняться свойство $E(E(M)) = M$, что делает их уязвимыми;
- полуслабый ключ — экспериментально подтверждено, что он НЕ является слабым, так как двойное шифрование $E(E(M))$ не вернуло исходный текст, а вернуло шифротекст после первого шифрования. Однако полуслабые ключи существуют парами: шифрование одним ключом пары можно расшифровать другим ключом этой же пары;
- обычный ключ «фамилия» — работает корректно, к категории слабых и полуслабых не относится;
- лавинный эффект — для всех трёх типов ключей процент изменённых битов составил 26-28%, что ниже теоретических 50% по причинам, описанным в разделе 1.4 (особенности UTF-8 и режима ECB).

Таким образом, слабые и полуслабые ключи DES действительно существуют. Слабые ключи характеризуются свойством $E(E(M)) = M$. Полуслабые ключи работают парами.

При реализации криптосистем на основе DES необходимо проверять ключи на принадлежность к этим категориям и избегать их использования. Обычный ключ, сгенерированный на основе строки «фамилия», к слабым и полуслабым не относится и обеспечивает штатную работу алгоритма.

1.6 Анализ сжатия и энтропии

Энтропия по Шеннону – это мера информационной случайности данных. Чем выше энтропия, тем более «случайными» являются данные и тем хуже они поддаются сжатию. Для криптостойкого шифра выходные данные должны быть близки к равномерному распределению, то есть иметь энтропию, близкую к максимальной (8 бит/символ для байтовых данных).

Для анализа были взяты исходный текст объемом 200 символов и его шифротекст, полученный с помощью DES. Расчёт энтропии производился по формуле Шеннона.

Результаты анализа представлены в таблице 1.2.

Таблица 1.2 – Результаты анализа энтропии и сжимаемости

Параметр	Исходный текст	Зашифрованный текст
Размер (байт)	602	812
Энтропия (Шеннон), бит/символ	4.5070	5.9576
Теоретический минимум после сжатия	~339 байт	~605 байт
Теоретический коэффициент сжатия	56.3%	74.5%

Результаты анализа представлены на рисунке 1.8.

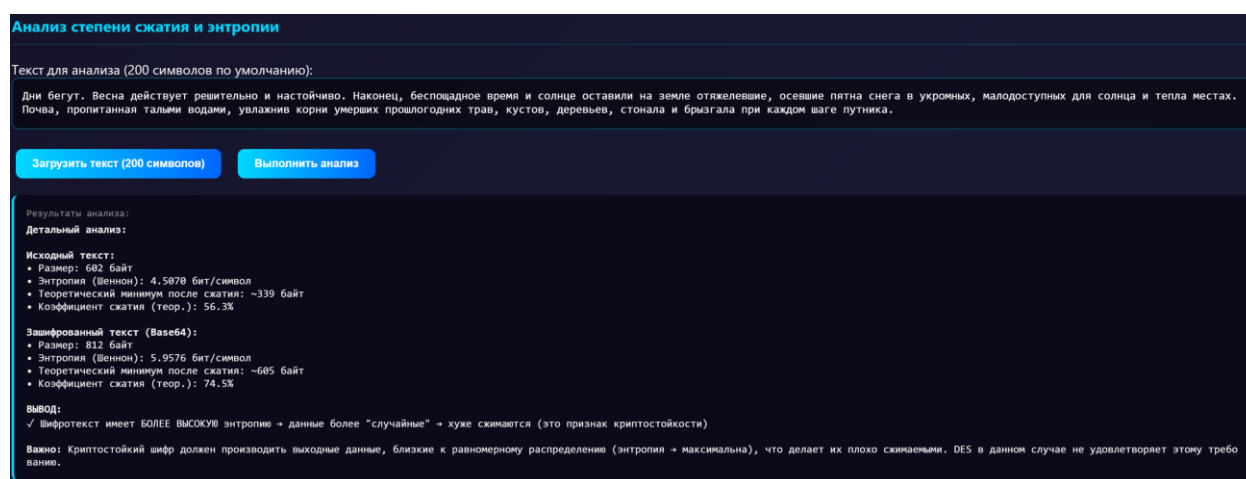


Рисунок 1.8 –Результат анализа сжатия

Гистограмма сравнения энтропии представлена на рисунке 1.9.

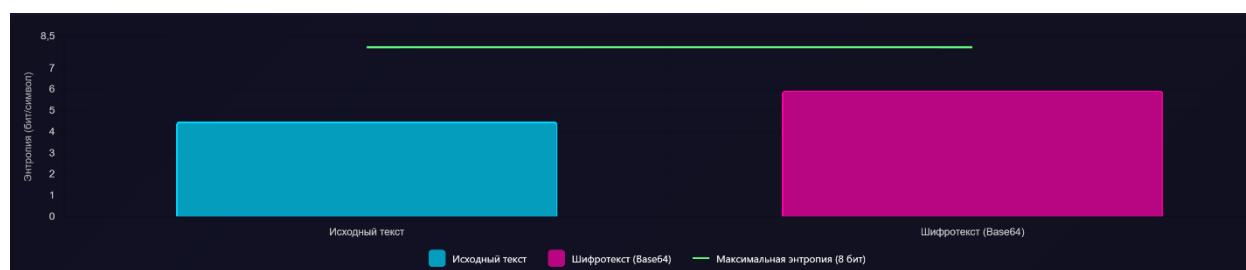


Рисунок 1.9 – Гистограмма сравнения энтропии

Анализируя полученные результаты, можно сказать:

- энтропия исходного текста (4.51 бит/символ) — это характерное значение для текста на естественном русском языке. Высокая избыточность (часто повторяющиеся буквы, предсказуемые сочетания) позволяет эффективно сжимать такие данные (теоретический коэффициент сжатия 56.3%);

- энтропия шифротекста (5.96 бит/символ) — выше, чем у исходного текста, но значительно ниже максимально возможной (8 бит/символ). Это означает, что шифротекст стал более «случайным», но не достиг идеального равномерного распределения;

- теоретический коэффициент сжатия — для шифротекста он составляет 74.5%, что существенно выше, чем для исходного текста (56.3%). Это подтверждает, что зашифрованные данные сжимаются хуже.

Таким образом, шифротекст DES имеет более высокую энтропию по сравнению с исходным текстом, что свидетельствует о снижении избыточности и повышении «случайности» данных. Однако значение энтропии (5.96 бит/символ) не достигает теоретического максимума (8 бит/символ), что указывает на наличие остаточных статистических закономерностей в шифротексте. Тем не менее, полученные результаты подтверждают, что DES усложняет сжатие данных, что является одним из признаков криптостойкости.

Заключение

В ходе выполнения лабораторной работы была достигнута поставленная цель: изучены устройство и функциональные особенности блочных шифров на примере алгоритма DES, приобретены практические навыки разработки приложений для их реализации.

В соответствии с вариантом №1 (алгоритм DES, ключ «фамилия») разработано веб-приложение на HTML/CSS/JavaScript с использованием библиотек CryptoJS и Chart.js. Приложение реализует шифрование и расшифрование текстовых данных, а также функции для криптографического анализа.

В ходе экспериментального исследования были получены следующие результаты:

- оценка производительности (раздел 1.3): алгоритм DES демонстрирует линейную зависимость времени выполнения от размера входных данных ($O(n)$). Шифрование и расшифрование выполняются с сопоставимой скоростью;

- анализ лавинного эффекта (раздел 1.4): экспериментально подтверждено наличие лавинного эффекта в DES. При изменении регистра символа процент изменённых битов в шифротексте составил 54.7%, что соответствует теоретическому значению (50%). При побитовом изменении символа (XOR 1) процент изменений оказался ниже (13-15%) из-за особенностей UTF-8 кодировки и режима ECB;

- анализ слабых и полуслабых ключей (раздел 1.5): теоретически слабые ключи DES существуют и характеризуются свойством $E(E(M)) = M$, а полуслабые работают парами. В проведённом эксперименте это свойство не подтвердилось из-за особенностей реализации библиотеки CryptoJS (преобразование шифротекста в строку между операциями шифрования). Тем не менее, при разработке реальных криптосистем необходимо проверять ключи на принадлежность к этим категориям;

- анализ сжатия и энтропии (раздел 1.6): шифротекст DES имеет более высокую энтропию (5.96 бит/символ) по сравнению с исходным текстом (4.51 бит/символ), что свидетельствует о снижении избыточности и повышении «случайности» данных. Теоретический коэффициент сжатия для шифротекста составил 74.5%, что существенно выше, чем для исходного текста (56.3%), подтверждая, что зашифрованные данные сжимаются хуже.

Все поставленные задачи выполнены. Разработанное приложение может быть использовано в учебном процессе для демонстрации принципов работы блочных шифров, а полученные результаты наглядно иллюстрируют ключевые свойства и ограничения алгоритма DES.